Project Implementation Report: KeyFort

Zero-Knowledge Encrypted Password Manager & Ephemeral Secret Sharing

DEVELOPER	PROJECT TYPE	ARCHITECTURE	REPOSITORY
MD SAMEER	Security & Mobile Application	3-Tier (Core + Backend + Android)	github.com/MohammedSameer-57/KeyFort

1. PROJECT OVERVIEW

KeyFort is a secure password management system built using a zero-knowledge security model. All encryption and decryption operations take place directly on the user's Android device. The remote server stores only encrypted data blocks and never receives the user's master password, raw encryption keys, or readable credentials.

2. CORE SYSTEM MODULES

keyfort-core

A pure Java 17 library containing all cryptographic logic: key derivation using PBKDF2 (600,000 rounds), symmetric vault encryption using AES-256-GCM, emergency recovery kits using RSA-3072, and SHA-256 ledger chaining.

keyfort-server

A Java 21 / Spring Boot 3 backend service that provides REST APIs for multi-device registration, user authorization with JWT, encrypted ciphertext synchronization, and audit ledger verification.

keyfort-android

A native Android application built with Material Design 3. Features local credential management, on-device password generation, encrypted secret sharing, and automatic clipboard wiping.

3. STEP-BY-STEP SYSTEM WORKFLOW

Step	Feature	Operation Details
1	**Master Key Setup**	User enters a master password. The device derives a 256-bit key using PBKDF2-HMAC-SHA256 (600,000 iterations). The key remains in local memory and is never sent to the network.
2	**Emergency Recovery**	An offline RSA-3072 keypair is created at registration. The user saves the private key as a backup. If the master password is forgotten, this key can restore vault access without server assistance.
3	**Credential Encryption**	When an entry is added, the Android app encrypts the credentials using AES-256-GCM with a random 96-bit IV. The resulting ciphertext is synced to the Spring Boot server.
4	**Credential Decryption**	To view an entry, the device fetches the ciphertext and decrypts it locally. When a password is copied, an internal timer automatically wipes the clipboard clean after 30 seconds.
5	**KeyFort Send**	Users can generate temporary sharing links formatted as `keyfort-send://<ciphertext>#<key>`. Because the decryption key is located in the URL fragment (#), web servers never see the key.
6	**Ledger Verification**	Each synchronization event is recorded on the server in an append-only SHA-256 hash chain. Users can audit this ledger directly from the mobile app to verify data integrity.

4. KEY SECURITY PROTECTIONS

- **Zero-Knowledge Architecture:** The server never receives readable passwords. A server compromise reveals only encrypted data blocks.
- **No Phone Number Dependency:** Account recovery uses an offline cryptographic key instead of SMS, protecting users against SIM-swapping attacks.
- **Screen Protection (FLAG_SECURE):** The mobile application blocks screenshots and screen recordings, showing only black pixels to unauthorized recording apps.
- **Automatic Clipboard Clearing:** Prevents other applications from accessing copied passwords after 30 seconds.
- **Cryptographic Audit Ledger:** Uses SHA-256 block chaining to ensure that synchronization events cannot be modified or deleted without detection.

5. TECHNOLOGY STACK

Layer	Technologies	Function
Core Cryptography	Java 17, BouncyCastle, JUnit 5	Key derivation, AES-256-GCM, RSA-3072 OAEP, ledger logic
Backend Service	Java 21, Spring Boot 3, Spring Security, JPA, H2	REST APIs, JWT issuance, device management, ledger audit
Mobile Client	Android SDK (minSdk 26, targetSdk 34), Material 3, Room, OkHttp 4	User interface, local SQLite cache, network synchronization
Tooling & Build	Gradle 8.10, Android Emulator, ADB, Git	Multi-module build orchestration, deployment, version control

6. PROJECT STATUS AND VERIFICATION

Current Implementation Status:
• **Unit Tests:** All test suites for `keyfort-core` and `keyfort-server` passed with 100% success.
• **Android Deployment:** Debug APK was built, installed on an Android emulator (API 34), and tested for registration, local caching, password generation, and ledger audit.
• **Version Control:** The complete source code is committed and pushed to GitHub under author **MD SAMEER** at `https://github.com/MohammedSameer-57/KeyFort`.